
Cross-Language Probe Invariance Depends on Readout Choice

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Cross-language probe transfer is often interpreted as evidence that a code model
2 represents program semantics independently of programming language. We
3 test this interpretation for variable roles in Qwen2.5-Coder-1.5B using parallel
4 Python, JavaScript, and PHP solutions. A masked character n -gram classifier
5 loses 0.126 macro-F1 on transfers involving Python relative to JavaScript $\leftrightarrow$ PHP.
6 A residual-stream probe changes by only -0.012 (95% problem-clustered interval
7 $[-0.034, 0.020]$), which appears to support language-invariant role encoding. This
8 probe mean-pools the occurrence’s own tokens, whereas the surface classifier masks
9 the identifier. Excluding the occurrence from the readout reduces mean transfer
10 from 0.844 to 0.589 and changes the boundary effect to -0.060 $[-0.088, -0.030]$.
11 The shift is -0.048 $[-0.088, -0.016]$. Context-matched untrained networks show
12 boundaries of -0.064 and -0.035 across two weight initializations; under nei-
13 ther is the trained probe flatter. Training-associated lift remains positive in both
14 language groups under both floors. Thus the original invariance is not identified
15 independently of the readout, while the restricted readout still contains training-
16 dependent role information.

17 1 Introduction

18 Linear probes measure whether a label is decodable from a representation. They do not establish that
19 the model uses the decoded feature, and their performance can reflect probe capacity or information
20 already available in the probe input [Hewitt and Liang, 2019, Voita and Titov, 2020, Ravichander
21 et al., 2021, Elazar et al., 2021, Belinkov, 2022]. Cross-language transfer adds a further interpretation.
22 If a probe fitted on one programming language transfers to another, the decoded feature may appear
23 independent of surface form.

24 Existing work has recovered code syntax across languages with probe baselines [Hernández López
25 et al., 2026], separated language-specific from language-agnostic embedding components [Utpala
26 et al., 2024], and measured uneven alignment among programming languages on parallel pro-
27 grams [Kargaran et al., 2025]; within-language suites map what code models encode across many
28 probe tasks [Karmakar and Robbes, 2021, Troshin and Chirkova, 2022, Karmakar and Robbes, 2024].
29 We do not add a task. These studies establish that code representations contain both shared and
30 language-specific structure. They do not provide a model-free, feature-restricted comparator on the
31 same cross-language prediction task.

32 We add that comparator for variable roles such as accumulator and index key [Sajaniemi, 2002].
33 Variable roles describe what a variable does, which makes them a plausible target for language-
34 independent representation. The initial probe result supports that interpretation. It changes when the
35 readout no longer pools the occurrence tokens that carry the identifier, which parallel implementations
36 often preserve [Kargaran et al., 2025]. This ablation localizes the instability to the readout support. It

Table 1: Cross-language role transfer averaged over three roles. *Close* contains JavaScript $\leftrightarrow$ PHP. *Python* contains the four directions to or from Python. Probe floors use the corresponding readout.

	close	Python	effect
Surface n -gram, name masked (no model)	0.929	0.804	-0.126
Probe, span-pooled (reads the name)	0.852	0.840	-0.012
Probe, occurrence-excluded	0.630	0.569	-0.060
untrained, occurrence-excluded	0.527	0.464	-0.064

does not, by itself, identify lexical identity as the cause because pooling support and token position change together.

2 Setup

Corpus. XLCOST [Zhu et al., 2022] provides parallel solutions keyed by problem identifiers that are stable across languages. Python, JavaScript, and PHP are the only three languages with at least 500 shared problems for every pair. For example, C# and Java share only 175 problems (Appendix B). We first restrict each language to the 869 problems present in all three. Role extraction then determines which of those problems contain a usable occurrence for each role, so role-specific coverage still differs across cells. Data splits use a fixed hash of the problem identifier and never separate occurrences from the same problem.

The Python data retain formatting that the JavaScript and PHP data do not. Because this formatting difference coincides with the language contrast, we remove whitespace before constructing character features. Equalizing whitespace on both sides changes each of the three audited iterator transfers by less than 0.001.

Conditions. We evaluate four conditions on matched language pairs.

- **Surface n -gram.** A character n -gram classifier over the context around an occurrence with every instance of the variable’s name replaced by a placeholder. The classifier is fitted on the source language and evaluated on the target. We report one fixed masked-window feature set rather than selecting the best feature set separately for each cell.
- **Probe, span-pooled.** Logistic regression on residual activations mean-pooled over the occurrence’s own tokens, layer chosen on source-language validation data. This reads the identifier.
- **Probe, occurrence-excluded.** The same, pooling the 16 tokens either side of the occurrence while excluding its tokens. Model inputs and weights are unchanged. Only the readout is altered.
- **Untrained.** The same architecture and tokenizer with randomly initialized weights, evaluated at each pooling. Randomly initialized networks can retain substantial lexical and structural information, so they provide a necessary floor [Zhang and Bowman, 2018, Hewitt and Liang, 2019]. Each probe is compared with the floor using the same readout.

Models are Qwen2.5-Coder-1.5B [Hui et al., 2024] and, for the untrained control, the corresponding Qwen2.5 architecture [Qwen et al., 2024]. Qwen2.5-Coder is continued-pretrained from the Qwen2.5 model family [Hui et al., 2024]. Reported uncertainty resamples complete problems rather than individual occurrences.

3 Surface transfer tracks the language contrast

The surface classifier reaches 0.929 between JavaScript and PHP without a model or identifier access, then falls to 0.804 on transfers involving Python. The table is an unweighted mean over cells. A separate pooled, problem-clustered estimate is -0.114 with a 95% interval of [-0.147, -0.081]. The aggregate is uneven across roles. Iterator contributes -0.249, compared with -0.064 for accumulator

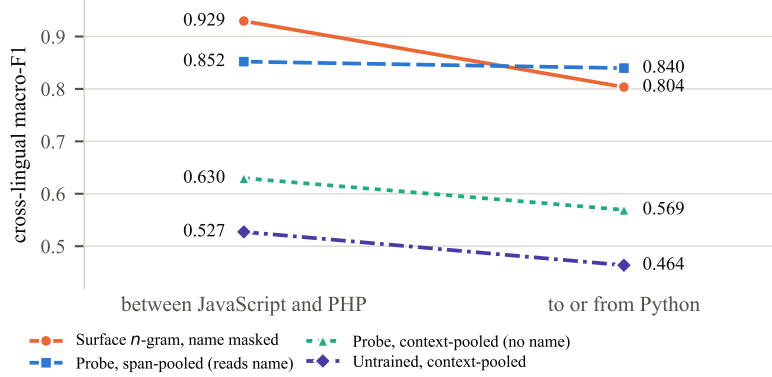


Figure 1: The surface baseline changes across the language contrast. The span-pooled probe appears stable but reads the occurrence tokens. Excluding the occurrence lowers probe transfer and yields a slope close to the matched untrained floor.

75 and index_key together. Iterator labels are also the least comparable because Python uses an AST-
 76 based extractor while JavaScript and PHP use regular expressions. Some cells contain only ten
 77 minority examples. We therefore treat the surface contrast as descriptive evidence for this extracted
 78 sample, not as a clean estimate of a language-family effect.

79 4 The inferred boundary depends on readout support

80 The span-pooled probe reaches 0.844 overall. Its cross-language boundary is -0.012 with a problem-
 81 clustered interval of $[-0.034, 0.020]$. This is the result that would normally be interpreted as
 82 language-independent role decodability.

83 Excluding the occurrence from the readout lowers mean transfer from 0.844 to 0.589 (Figure 1). The
 84 context-matched untrained model reaches 0.485. The boundary changes to -0.060 $[-0.088, -0.030]$;
 85 the readout-induced shift is -0.048 $[-0.088, -0.016]$. The original flat boundary is therefore not
 86 robust to where activations are read.

87 The occurrence-excluded untrained model has a -0.064 boundary $[-0.089, -0.039]$; subtracting
 88 this floor gives a trained-minus-untrained boundary of $+0.003$ $[-0.027, 0.036]$. A second weight
 89 initialization yields a -0.035 floor boundary with an overall level of 0.497 against the first initializa-
 90 tion’s 0.485: the floor’s level is stable while its boundary varies across initializations, and the trained
 91 boundary (-0.060) lies within that variation, at its steeper end. Under neither initialization is the
 92 trained probe flatter than the untrained network. Training still contributes: trained-minus-untrained
 93 transfer is $+0.102$ $[0.065, 0.132]$ close and $+0.106$ $[0.081, 0.127]$ for Python transfers against the
 94 first floor, and $+0.110$ and $+0.083$ in point terms against the second.

95 Identifier exclusion also matters within languages. Mean in-language F1 falls from 0.898 to 0.771, a
 96 reduction of 0.127. The context-pooled untrained model reaches 0.623 in language. Direct access
 97 to the occurrence therefore accounts for a substantial part of both in-language and cross-language
 98 decodability.

99 The remaining occurrence-excluded margin supports training-dependent role information in this
 100 readout. It does not establish a language-independent mechanism because the training lift is similar
 101 on both sides of the boundary. The surface baseline outperforms the context-pooled probe in 14
 102 of 18 matched cells with problem-clustered intervals below zero for the paired difference. This
 103 shows surface sufficiency on the same prediction items; it is not a decomposition of the model signal,
 104 because the model still processes unmasked source code.

105 5 Discussion

106 The central result concerns measurement. A probe that pools over an occurrence can use information
 107 concentrated at those tokens, including lexical regularities preserved by parallel programs. A

randomly initialized model tests what training contributes under a fixed readout, not whether that readout isolates the intended construct. Conversely, occurrence exclusion changes the pooled positions and their information content together. Calling the difference “identifier leakage” would therefore overstate what this experiment identifies.

The model-free condition exposes a different source of evidence. Its input withholds every target-name occurrence, while the probe model sees the original program. The two predictors share labels and test items but not information sets, so their comparison establishes surface sufficiency rather than an isolated representational increment. Model-free baselines are established in code probing generally [Troshin and Chirkova, 2022], but the cited cross-language studies do not fit one on the same prediction items [Hernández López et al., 2026, Utpala et al., 2024, Kargaran et al., 2025]. The present result extends the usual distinction between decodability and model use [Elazar et al., 2021, Belinkov, 2022]. Decodability can also arise from a label correlate exposed directly by the readout.

Reconciling with prior alignment results. Kargaran et al. [2025] find Python the *least* aligned of these three languages on parallel programs, whereas the span-pooled probe changes little across transfers involving Python. The studies measure different quantities, so agreement is not required. Nevertheless, excluding the identifier produces a probe contrast in the direction of their alignment result. This convergence is compatible with a genuine Python boundary, but it does not establish a common mechanism.

Falsifying evidence. Our interpretation would weaken if a context-pooled probe developed a growing margin over its matched floor at larger scale. It would also weaken if same-readout input-level identifier replacement preserved the flat transfer curve, or if a parallel corpus without shared naming conventions produced the original span-pooled invariance. These experiments distinguish lexical leakage, readout choice, and genuinely shared role information.

Implication for linguistic medium studies. When linguistic or symbolic form is manipulated, the measurement instrument must not retain a feature that the manipulation preserves. Parallel corpora concentrate such features, including names, entities, and numbers. A feature-restricted model-free comparator and a matched readout ablation provide two inexpensive checks before invariance is attributed to a model representation.

Limitations. The study uses one 1.5B model, three languages, three roles, and a single context width. Python supplies the only broad language contrast. Role-specific coverage differs after extraction even though the corpus is restricted to a common problem pool, so composition may still affect group means. Iterator labels use different extractor families across languages and carry most of the surface effect; no multilingual hand-audit quantifies extractor error. The occurrence-excluded control uses two random weight initializations at two probe seeds each, whereas trained probes use five seeds; its boundary varies across initializations (-0.064 , -0.035) while its level does not (0.485 , 0.497), and only the first initialization’s interval estimates are computed. It removes the target occurrence from the readout but may retain correlated nearby uses and names, and it changes pooling support rather than intervening on lexical identity alone. No same-readout probe under input-level identifier replacement is available. These results concern decodability rather than model use and do not support a causal claim about the model’s computation.

References

- Yonatan Belinkov. Probing classifiers: Promises, shortcomings, and advances. *Computational Linguistics*, 48(1):207–219, 2022. doi: 10.1162/coli_a_00422.
- Yanai Elazar, Shauli Ravfogel, Alon Jacovi, and Yoav Goldberg. Amnesic probing: Behavioral explanation with amnesic counterfactuals. *Transactions of the Association for Computational Linguistics*, 9:160–175, 2021. doi: 10.1162/tacl_a_00359.
- José Antonio Hernández López, Martin Weyssow, Jesús Sánchez Cuadrado, and Houari Sahraoui. Syntactic multilingual probing of pre-trained language models of code. *Journal of Systems and Software*, 2026. doi: 10.1016/j.jss.2025.112604.

- 157 John Hewitt and Percy Liang. Designing and interpreting probes with control tasks. In *Proceedings*
158 *of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th*
159 *International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2733–
160 2743, Hong Kong, China, 2019. Association for Computational Linguistics. doi: 10.18653/v1/
161 D19-1275.
- 162 Binyuan Hui, Jian Yang, Zeyu Cui, Jiayi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang,
163 Bowen Yu, Keming Lu, Kai Dang, Yang Fan, Yichang Zhang, An Yang, Rui Men, Fei Huang,
164 Bo Zheng, Yibo Miao, Shanghaoran Quan, Yunlong Feng, Xingzhang Ren, Xuancheng Ren,
165 Jingren Zhou, and Junyang Lin. Qwen2.5-coder technical report. *arXiv preprint arXiv:2409.12186*,
166 2024. doi: 10.48550/arXiv.2409.12186.
- 167 Amir Hossein Kargaran, Yihong Liu, François Yvon, and Hinrich Schuetze. How programming
168 concepts and neurons are shared in code language models. In *Findings of the Association for*
169 *Computational Linguistics: ACL 2025*, 2025. doi: 10.18653/v1/2025.findings-acl.1379.
- 170 Anjan Karmakar and Romain Robbes. What do pre-trained code models know about code? In
171 *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages
172 1332–1336, Melbourne, Australia, 2021. IEEE. doi: 10.1109/ASE51524.2021.9678927.
- 173 Anjan Karmakar and Romain Robbes. INSPECT: Intrinsic and systematic probing evaluation for
174 code transformers. *IEEE Transactions on Software Engineering*, 50(2):220–238, 2024. doi:
175 10.1109/TSE.2023.3341624.
- 176 Qwen, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan
177 Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang,
178 Jianxin Yang, Jiayi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin
179 Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi
180 Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan,
181 Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report. *arXiv preprint*
182 *arXiv:2412.15115*, 2024.
- 183 Abhilasha Ravichander, Yonatan Belinkov, and Eduard Hovy. Probing the probing paradigm: Does
184 probing accuracy entail task relevance? In *Proceedings of the 16th Conference of the European*
185 *Chapter of the Association for Computational Linguistics: Main Volume*, pages 3363–3377, Online,
186 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.eacl-main.295.
- 187 Jorma Sajaniemi. An empirical analysis of roles of variables in novice-level procedural programs. In
188 *Proceedings IEEE 2002 Symposia on Human Centric Computing Languages and Environments*,
189 pages 37–39, Arlington, VA, USA, 2002. IEEE Computer Society. doi: 10.1109/HCC.2002.
190 1046340.
- 191 Sergey Troshin and Nadezhda Chirkova. Probing pretrained models of source codes. In *Pro-*
192 *ceedings of the Fifth BlackboxNLP Workshop on Analyzing and Interpreting Neural Networks*
193 *for NLP*, pages 371–383, Abu Dhabi, United Arab Emirates (Hybrid), December 2022. As-
194 sociation for Computational Linguistics. doi: 10.18653/v1/2022.blackboxnlp-1.31. URL
195 <https://aclanthology.org/2022.blackboxnlp-1.31/>.
- 196 Saiteja Utpala, Alex Gu, and Pin-Yu Chen. Language agnostic code embeddings. In *Proceedings*
197 *of the 2024 Conference of the North American Chapter of the Association for Computational*
198 *Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 2024. doi: 10.18653/v1/
199 2024.naacl-long.38.
- 200 Elena Voita and Ivan Titov. Information-theoretic probing with minimum description length. In
201 *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*
202 *(EMNLP)*, 2020. doi: 10.18653/v1/2020.emnlp-main.14.
- 203 Kelly W. Zhang and Samuel R. Bowman. Language modeling teaches you more than translation
204 does: Lessons learned through auxiliary syntactic task analysis. In *Proceedings of the 2018*
205 *EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, pages
206 359–361, Brussels, Belgium, November 2018. Association for Computational Linguistics. doi:
207 10.18653/v1/W18-5448.

208 Ming Zhu, Aneesh Jain, Karthik Suresh, Roshan Ravindran, Sindhu Tipirneni, and Chandan K.
209 Reddy. XLCoS^T: A benchmark dataset for cross-lingual code intelligence. *arXiv preprint*
210 *arXiv:2206.08474*, 2022. doi: 10.48550/arXiv.2206.08474.

211 A Complete transfer results

Table 2: All eighteen transfer cells for Qwen2.5-Coder-1.5B. *Surface* is the strongest masked-context n -gram feature; *name* uses the variable’s name alone; *probe* is the residual-stream probe. Majority and shuffled-label controls sit near chance throughout. ρ is a descriptive one-instance score step, not a confidence interval or hypothesis test.

role	pair	n	surface	name	probe	major.	shuf.	ρ
accumulator	JavaScript→PHP	1,976	0.951	0.897	0.904	0.361	0.470	0.0005
accumulator	JavaScript→Python	3,341	0.813	0.873	0.875	0.415	0.473	0.0004
accumulator	PHP→JavaScript	1,903	0.954	0.897	0.918	0.372	0.490	0.0005
accumulator	PHP→Python	1,377	0.782	0.824	0.833	0.276	0.439	0.0008
accumulator	Python→JavaScript	3,294	0.785	0.874	0.918	0.370	0.504	0.0003
accumulator	Python→PHP	1,478	0.736	0.845	0.874	0.271	0.466	0.0007
index_key	JavaScript→PHP	1,976	0.940	0.834	0.897	0.384	0.478	0.0005
index_key	JavaScript→Python	3,341	0.859	0.787	0.883	0.330	0.497	0.0003
index_key	PHP→JavaScript	1,903	0.964	0.838	0.880	0.376	0.516	0.0005
index_key	PHP→Python	1,377	0.839	0.782	0.854	0.387	0.515	0.0008
index_key	Python→JavaScript	3,294	0.871	0.822	0.915	0.314	0.470	0.0003
index_key	Python→PHP	1,478	0.858	0.838	0.874	0.419	0.459	0.0008
iterator	JavaScript→PHP	1,976	0.937	0.862	0.906	0.448	0.488	0.0008
iterator	JavaScript→Python	3,341	0.774	0.733	0.939	0.444	0.472	0.0005
iterator	PHP→JavaScript	1,903	0.965	0.873	0.851	0.446	0.478	0.0008
iterator	PHP→Python	1,377	0.576	0.788	0.865	0.428	0.462	0.0010
iterator	Python→JavaScript	3,294	0.790	0.769	0.918	0.466	0.484	0.0007
iterator	Python→PHP	1,478	0.741	0.788	0.902	0.475	0.438	0.0020

212 B Pairwise problem overlap in XLCoS^T

213 C Full masked-probe results

214 D Renaming and formatting audits

215 E Transfer-mechanism diagnostics

216 F Language-geometry controls

217 **The typing axis is not privileged.** Probing the same activations under every alternative 4-versus-3
218 partition of the seven languages ($\binom{7}{4} = 35$ groupings minus the real one, so 34 controls) gives
219 best-layer macro-F1 0.9959 on average (min 0.9918, max 1.0000), against 1.0000 for the true
220 static/dynamic split; 5 of the 34 controls match or exceed it. Any grouping of these languages is
221 near-perfectly decodable, so the decodability of the typing split is evidence that language identity is
222 encoded, not that type discipline is. At layer 4, where $|r|$ peaks at 0.941, the remaining components
223 carry almost none of the label: PC2 $r = -0.014$, PC3 $r = +0.026$, PC4 $r = +0.171$.

224 G Complete transfer heatmaps

225 **Reading the heatmaps.** Rows are source languages and columns are target languages. These plots
226 retain the complete available transfer matrices, including cells excluded from matched-pair inference
227 because problem overlap is too small.

Table 3: Problem-clustered intervals for every surface transfer cell. The final row is the Python-pair minus close-pair boundary effect.

role	pair	macro-F1	95% CI	problems
accumulator	JavaScript→PHP	0.925	[0.897, 0.946]	409
accumulator	JavaScript→Python	0.845	[0.812, 0.873]	389
accumulator	PHP→JavaScript	0.953	[0.929, 0.968]	409
accumulator	PHP→Python	0.844	[0.810, 0.873]	378
accumulator	Python→JavaScript	0.853	[0.815, 0.886]	389
accumulator	Python→PHP	0.861	[0.822, 0.891]	378
index_key	JavaScript→PHP	0.875	[0.823, 0.911]	409
index_key	JavaScript→Python	0.878	[0.836, 0.909]	389
index_key	PHP→JavaScript	0.931	[0.902, 0.954]	409
index_key	PHP→Python	0.846	[0.801, 0.882]	378
index_key	Python→JavaScript	0.887	[0.841, 0.919]	389
index_key	Python→PHP	0.881	[0.831, 0.915]	378
iterator	JavaScript→PHP	0.894	[0.808, 0.945]	409
iterator	JavaScript→Python	0.608	[0.554, 0.668]	389
iterator	PHP→JavaScript	0.972	[0.908, 0.994]	409
iterator	PHP→Python	0.564	[0.520, 0.621]	378
iterator	Python→JavaScript	0.848	[0.732, 0.916]	389
iterator	Python→PHP	0.813	[0.705, 0.891]	378
ALL	Python pairs — close pair	-0.114	[-0.147, -0.081]	432

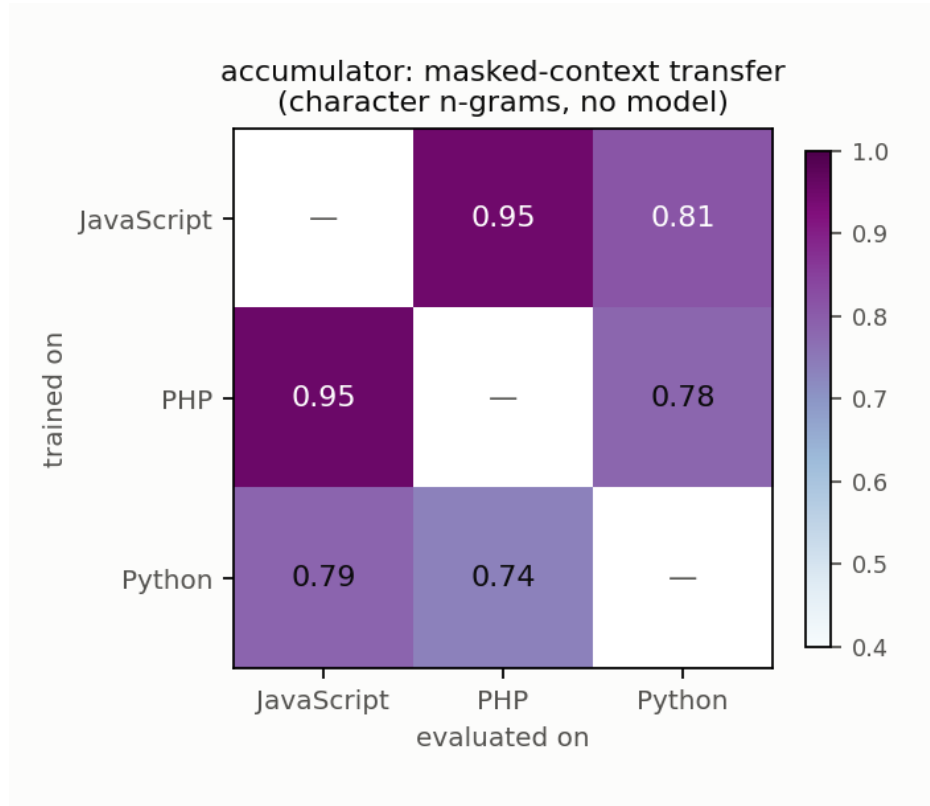


Figure 2: Available accumulator transfer matrix for Qwen2.5-Coder-1.5B. Cells without adequate matched problem overlap are descriptive only and are not used for the main boundary estimate.

Table 4: Probe transfer per model, averaged within each group. The untrained network shares Qwen2.5-1.5B’s architecture and tokenizer with randomly initialised weights; it is the floor a representational claim must clear, and it is itself flat.

model	close pair	Python pairs	all	vs. untrained
Qwen2.5-1.5B	0.899	0.888	0.892	+0.316
Qwen2.5-Coder-1.5B	0.893	0.887	0.889	+0.314
Qwen2.5-1.5B (untrained)	0.590	0.568	0.575	—

Table 5: Shared problem identifiers between every pair of XLCoST languages, train split. Matched cross-lingual transfer needs a pair to share enough problems to hold out a test fold; only the three bold cells clear 500. No pair outside Python/JavaScript/PHP does, including same-family pairs.

	C++	C#	Java	JavaScript	PHP	Python
C++	—	115	122	75	17	74
C#	115	—	175	75	14	62
Java	122	175	—	85	11	66
JavaScript	75	75	85	—	1,529	2,953
PHP	17	14	11	1,529	—	1,145
Python	74	62	66	2,953	1,145	—

H Reproducibility

Every figure is regenerated from committed result files, and a regression test recomputes the headline values from those files. The context-pooled trained and untrained conditions and the prediction-level boundary analysis are committed per cell; the second floor initialization’s per-cell artifacts are pending commitment from the verification run. Intervals resample stable problem identifiers once across all cells and conditions. Activation extraction is the only accelerator-dependent step. Probes use logistic regression on residual activations pooled over either the occurrence or the 16 tokens on each side. The surface baseline uses a `char_wb` tf-idf vectorizer with $n \in [2, 4]$ and source-language logistic regression. Store identities record model initialization and pooling so conditions cannot be mixed silently.

Table 6: Per-cell macro-F1 for the four intersection-sample conditions. *Lift* subtracts the context-matched randomly initialized result from the context-pooled trained result.

role	pair	surface	span	context	random	lift
accumulator	JavaScript→PHP	0.928	0.833	0.551	0.516	+0.035
accumulator	PHP→JavaScript	0.952	0.836	0.640	0.582	+0.058
accumulator	JavaScript→Python	0.835	0.840	0.506	0.513	-0.008
accumulator	PHP→Python	0.831	0.809	0.554	0.424	+0.130
accumulator	Python→JavaScript	0.858	0.866	0.600	0.467	+0.133
accumulator	Python→PHP	0.865	0.854	0.476	0.446	+0.030
index_key	JavaScript→PHP	0.881	0.812	0.696	0.513	+0.183
index_key	PHP→JavaScript	0.935	0.824	0.659	0.568	+0.092
index_key	JavaScript→Python	0.875	0.805	0.661	0.556	+0.105
index_key	PHP→Python	0.858	0.746	0.594	0.370	+0.224
index_key	Python→JavaScript	0.879	0.862	0.672	0.520	+0.151
index_key	Python→PHP	0.878	0.828	0.576	0.328	+0.249
iterator	JavaScript→PHP	0.901	0.950	0.563	0.494	+0.069
iterator	PHP→JavaScript	0.980	0.859	0.668	0.491	+0.177
iterator	JavaScript→Python	0.577	0.836	0.523	0.473	+0.050
iterator	PHP→Python	0.548	0.780	0.517	0.514	+0.003
iterator	Python→JavaScript	0.830	0.966	0.597	0.476	+0.121
iterator	Python→PHP	0.810	0.885	0.556	0.477	+0.080

Table 7: Problem-clustered paired differences between the context-pooled probe and the surface comparator. Negative values favor the surface comparator.

role	pair	Δ	CI low	CI high
accumulator	JavaScript→PHP	-0.315	-0.403	-0.214
accumulator	JavaScript→Python	-0.253	-0.333	-0.178
accumulator	PHP→JavaScript	-0.302	-0.405	-0.211
accumulator	PHP→Python	-0.177	-0.269	-0.083
accumulator	Python→JavaScript	-0.243	-0.335	-0.150
accumulator	Python→PHP	-0.337	-0.443	-0.196
index_key	JavaScript→PHP	-0.069	-0.177	+0.034
index_key	JavaScript→Python	-0.159	-0.306	-0.051
index_key	PHP→JavaScript	-0.131	-0.240	-0.039
index_key	PHP→Python	-0.158	-0.297	-0.023
index_key	Python→JavaScript	-0.114	-0.200	-0.037
index_key	Python→PHP	-0.318	-0.410	-0.239
iterator	JavaScript→PHP	-0.346	-0.507	+0.000
iterator	JavaScript→Python	-0.135	-0.301	-0.031
iterator	PHP→JavaScript	-0.428	-0.508	-0.001
iterator	PHP→Python	-0.074	-0.225	+0.001
iterator	Python→JavaScript	-0.162	-0.392	+0.018
iterator	Python→PHP	-0.190	-0.346	-0.001

Table 8: Problem-clustered uncertainty for the aggregate boundary contrasts. The effect is Python-transfer minus JavaScript↔PHP macro-F1. The difference-in-differences subtracts the untrained boundary from the trained boundary. Intervals resample problem identifiers and are conditional on the committed model runs, including one random weight initialization.

estimand	estimate	95% CI
span-pooled trained boundary	-0.012	[-0.034, +0.020]
occurrence-excluded trained boundary	-0.060	[-0.088, -0.030]
occurrence-excluded untrained boundary	-0.064	[-0.089, -0.039]
trained — untrained boundary	+0.003	[-0.027, +0.036]
training lift, close pair	+0.102	[+0.065, +0.132]
training lift, Python pairs	+0.106	[+0.081, +0.127]
boundary shift after exclusion	-0.048	[-0.088, -0.016]

Table 9: Rolewise occurrence-excluded effects and floor-adjusted transfer. Effects are Python minus close-pair macro-F1; lift subtracts the matched untrained score. The final column is close-pair lift minus Python-transfer lift. These descriptive role summaries do not carry separate intervals.

role	trained effect	random effect	lift close	lift Python	Δ lift
accumulator	-0.062	-0.087	+0.046	+0.071	-0.025
index_key	-0.052	-0.097	+0.138	+0.182	-0.045
iterator	-0.067	-0.008	+0.123	+0.064	+0.060

Table 10: Aggregate diagnostics retained in the intersection-sample masked-probe export. Surface counts are test occurrences; probe counts are shared source–target problems. Surface rows have no in-domain or seed fields. The untrained condition has two probe seeds versus five for the trained conditions.

condition	transfer	in-domain	shuffled source	seeds	cell population
surface window	0.846	—	0.476	—	2,617–3,138 occ.
span-pooled trained	0.844	0.898	0.468	5	374–406 problems
context-pooled trained	0.589	0.771	0.448	5	374–406 problems
context-pooled untrained	0.485	0.623	0.465	2	374–406 problems

Table 11: Uncapped renaming audit for Python-source transfer. C1, C2, and C4 are the committed identifier-renaming conditions. These samples differ slightly from the frozen main analysis and are reported as a robustness audit rather than pooled with it.

role	condition	target	n_{test}	name	surface	majority
accumulator	C1	JavaScript	12,746	0.396	0.790	0.423
accumulator	C1	PHP	3,517	0.222	0.768	0.417
accumulator	C2	JavaScript	12,746	0.410	0.789	0.423
accumulator	C2	PHP	3,517	0.473	0.786	0.417
accumulator	C4	JavaScript	12,746	0.389	0.793	0.423
accumulator	C4	PHP	3,517	0.565	0.817	0.417
index_key	C1	JavaScript	12,746	0.528	0.845	0.443
index_key	C1	PHP	3,517	0.599	0.880	0.439
index_key	C2	JavaScript	12,746	0.506	0.871	0.443
index_key	C2	PHP	3,517	0.626	0.879	0.439
index_key	C4	JavaScript	12,746	0.523	0.878	0.443
index_key	C4	PHP	3,517	0.509	0.879	0.439
iterator	C1	JavaScript	12,746	0.512	0.662	0.491
iterator	C1	PHP	3,517	0.443	0.650	0.489
iterator	C2	JavaScript	12,746	0.449	0.555	0.491
iterator	C2	PHP	3,517	0.337	0.538	0.489
iterator	C4	JavaScript	12,746	0.468	0.624	0.491
iterator	C4	PHP	3,517	0.431	0.630	0.489

Table 12: Whitespace-normalisation audit for the three decisive iterator surface transfers. Scores are unchanged at the displayed precision.

pair	raw	normalised	Δ
PHP→JavaScript	0.9654	0.9654	+0.0000
PHP→Python	0.5756	0.5756	+0.0000
Python→JavaScript	0.7901	0.7901	+0.0000

Table 13: Transfer mechanism for the iterator role. *Surviving* is the share of the source classifier’s discriminative mass realised in the target; *flip* is the share of that mass whose sign reverses. Surviving mass is uncorrelated with transfer; flipped mass predicts it almost exactly. Source-weighted variants are given for robustness.

pair	F1	surviving	flip	agree	flip (src-wt.)
PHP→JavaScript	0.965	0.735	0.072	+0.895	0.072
JavaScript→PHP	0.937	0.698	0.052	+0.945	0.068
Python→JavaScript	0.790	0.862	0.178	+0.695	0.211
JavaScript→Python	0.774	0.757	0.232	+0.811	0.241
Python→PHP	0.741	0.696	0.280	+0.528	0.367
PHP→Python	0.576	0.697	0.366	+0.476	0.346

Table 14: Masking an entire syntactic character class on both sides of the transfer. No class accounts for more than 0.021 of the 0.39 gap, so the realignment is distributed rather than carried by block delimiters or statement terminators.

pair	masked class	macro-F1	Δ
PHP→JavaScript	terminator	0.9496	-0.0159
PHP→JavaScript	brace	0.9645	-0.0010
PHP→JavaScript	bracket	0.9711	+0.0057
PHP→JavaScript	paren	0.9609	-0.0045
PHP→JavaScript	operator	0.9441	-0.0213
PHP→Python	terminator	0.5786	+0.0030
PHP→Python	brace	0.5750	-0.0006
PHP→Python	bracket	0.5733	-0.0023
PHP→Python	paren	0.5947	+0.0191
PHP→Python	operator	0.5801	+0.0045

Table 15: Point-biserial correlation between PC1 of last-token residual activations and a static/dynamic typing label, by layer, with PC1’s explained-variance ratio. The sign of r is arbitrary (PCA fixes the axis only up to reflection, and each layer is refit independently); magnitude is what carries meaning.

layer	$ r $	EVR	layer	$ r $	EVR
0	0.883	0.481	15	0.894	0.309
1	0.898	0.447	16	0.882	0.303
2	0.914	0.391	17	0.874	0.296
3	0.939	0.373	18	0.873	0.289
4	0.941	0.370	19	0.873	0.299
5	0.862	0.369	20	0.903	0.291
6	0.828	0.361	21	0.886	0.276
7	0.844	0.342	22	0.859	0.265
8	0.896	0.339	23	0.842	0.252
9	0.919	0.333	24	0.812	0.242
10	0.880	0.327	25	0.774	0.253
11	0.893	0.302	26	0.694	0.255
12	0.917	0.292	27	0.732	0.237
13	0.904	0.314	28	0.576	0.242
14	0.898	0.309			

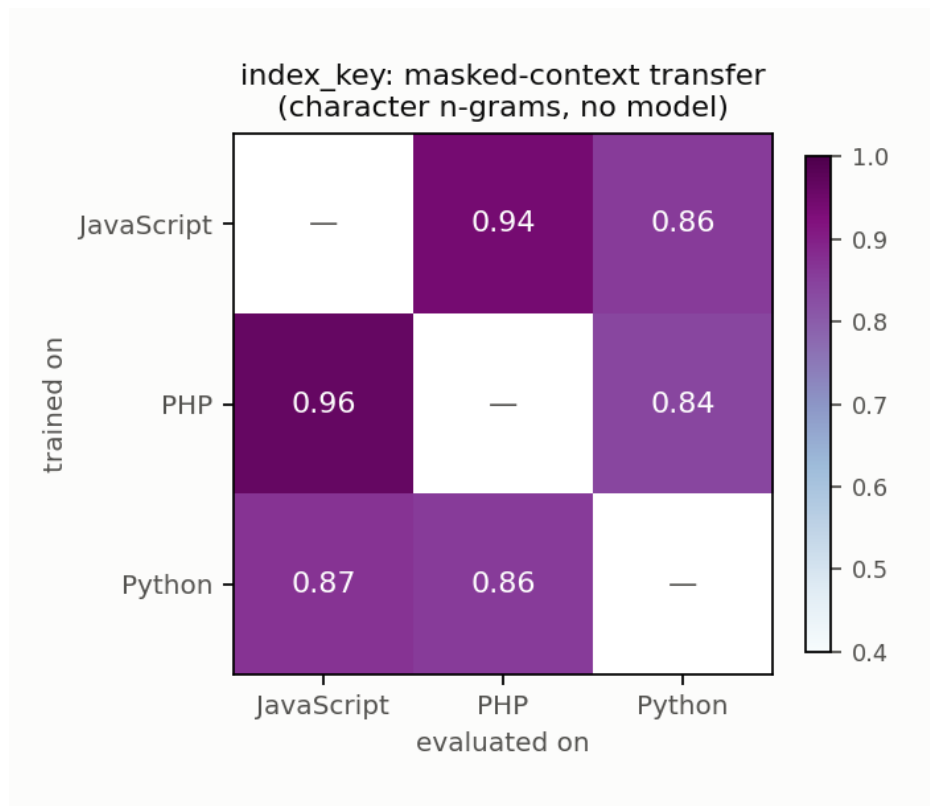


Figure 3: Available index/key transfer matrix for Qwen2.5-Coder-1.5B. Cells without adequate matched problem overlap are descriptive only and are not used for the main boundary estimate.

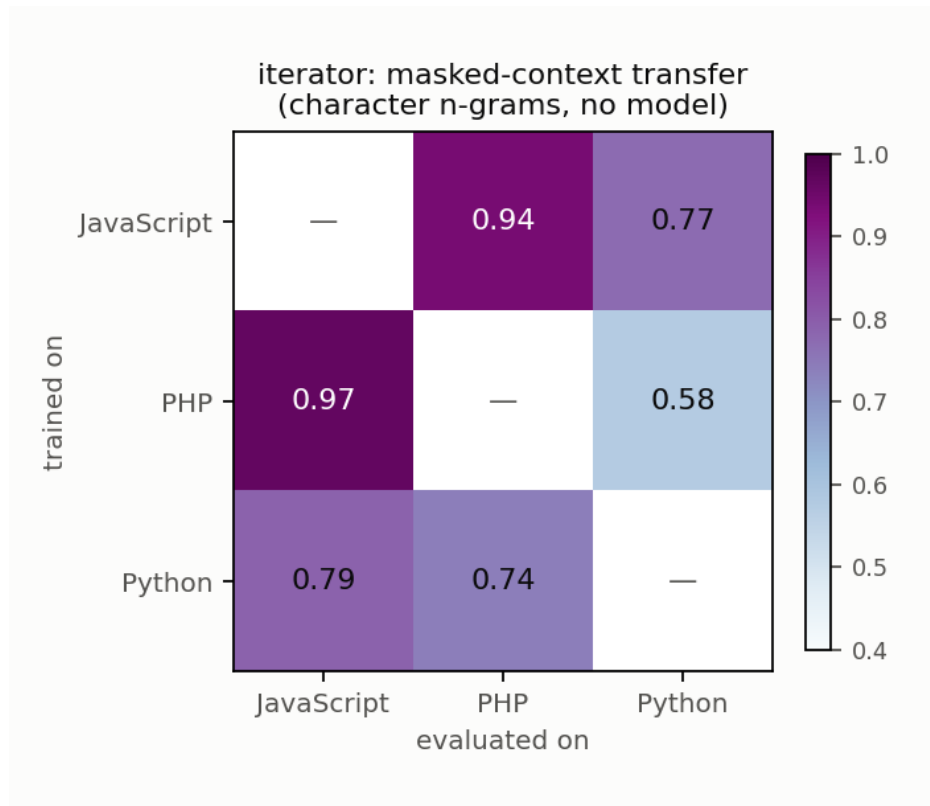


Figure 4: Available iterator transfer matrix for Qwen2.5-Coder-1.5B. Cells without adequate matched problem overlap are descriptive only and are not used for the main boundary estimate.

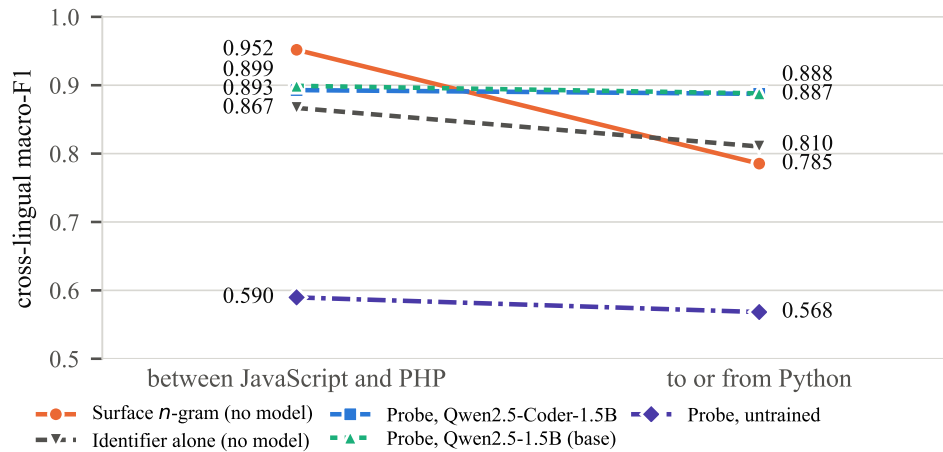


Figure 5: Span-pooled transfer contrast for the two trained models and the architecture-matched random-initialization floor. This capped-sample diagnostic is distinct from the intersection-sample context-pooled analysis.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and introduction state the identifier-access confound, the matched context-pooled comparison, and the resulting limit on language-invariance claims. The scope is restricted to one model, three languages, and three roles.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: The limitations paragraph covers model and language scope, role-specific coverage, extractor differences, random-seed counts, the mixed-sample span floor, residual identifier correlates, and the absence of causal evidence.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper reports no theoretical results.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The setup gives the corpus restriction, problem-level split policy, feature restrictions, pooling rules, model identifiers, and source-validation layer selection. The reproducibility appendix records classifier settings and artifact provenance.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: Result files and analysis scripts are prepared for release, but the manuscript does not yet provide an anonymous review URL or a complete archival package.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: The setup specifies the frozen problem-hash split, fixed masked feature set, pooling rules, context width, and source-validation layer selection. The reproducibility appendix gives the tf-idf and logistic-regression configuration.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Problem-clustered intervals are reported for the surface contrast, paired cell comparisons, readout-induced boundary shift, and trained-minus-untrained boundary. These intervals resample problem identifiers and remain conditional on the committed model runs, including one random weight initialization.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.

- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [No]

Justification: The paper identifies activation extraction as the accelerator-dependent step but does not report hardware, memory, or complete wall-clock costs.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The work analyses publicly released models on a public benchmark of competitive-programming solutions. It involves no human subjects, no personally identifying data, and no deployment.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [N/A]

Justification: The work is an analysis of how existing code models represent variable roles. We identify no societal impact specific to it beyond that of the public models and benchmark it studies.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper releases no data or models. It analyses publicly available ones.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [No]

Justification: XLCoST and the Qwen model releases are cited by name and identifier, but the manuscript does not enumerate their license names and terms.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.

- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [No]

Justification: The analysis scripts and result files have artifact provenance, but a complete user-facing release package and documentation are not yet included with the anonymous submission.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper involves no crowdsourcing and no human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

552 Justification: The paper involves no human subjects, so no IRB review was required.

553 Guidelines:

- 554 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
- 555 with human subjects.
- 556 • Depending on the country in which research is conducted, IRB approval (or equivalent)
- 557 may be required for any human subjects research. If you obtained IRB approval, you
- 558 should clearly state this in the paper.
- 559 • We recognize that the procedures for this may vary significantly between institutions
- 560 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
- 561 guidelines for their institution.
- 562 • For initial submissions, do not include any information that would break anonymity (if
- 563 applicable), such as the institution conducting the review.

564 **16. Declaration of LLM usage**

565 Question: Does the paper describe the usage of LLMs if it is an important, original, or

566 non-standard component of the core methods in this research? Note that if the LLM is used

567 only for writing, editing, or formatting purposes and does *not* impact the core methodology,

568 scientific rigor, or originality of the research, declaration is not required.

569 Answer: [N/A]

570 Justification: Large language models are the object of study, not an important or non-standard

571 component used to develop the research method. Writing and editing assistance is exempt

572 under the question.

573 Guidelines:

- 574 • The answer [N/A] means that the core method development in this research does not
- 575 involve LLMs as any important, original, or non-standard components.
- 576 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
- 577 be described.